

UNIVERSIDAD PERUANA UNIÓN

Facultad de Ingeniería y Arquitectura

EP Ingeniería de Sistemas

Desarrollo de Aplicaciones Móviles (DAM)

Ciclo VI · Semestre 2026-2

ACTIVIDAD AUTÓNOMA N.º 08

PRUEBAS INTEGRALES DE CRUD REST, TAXONOMÍA DE ERRORES Y PRUEBAS AUTOMATIZADAS MULTIPLATAFORMA

Estudiante / Autor	Julio Fabián Rodríguez Bazán
Repositorio GitHub	https://github.com/fabianrodriguez33/Pharma-Mobil
Sesión	Sesión 08 · Pruebas integrales de CRUD REST y taxonomía de errores
Rama	feature/ktor-client · https://github.com/fabianrodriguez33/Pharma-Mobil/tree/feature/ktor-client
Fecha de entrega	01/10/2026

Entorno y metodología

Todas las evidencias se obtuvieron el 01/10/2026 contra el backend PharmaSoft real (Spring Boot, Oracle Free en Docker, puerto 8080) y la aplicación PharmaMobil instalada en el emulador Pixel 8 (Android, 1080×2400), que accede al backend mediante `http://10.0.2.2:8080/`. Las respuestas de la matriz del Producto 1 provienen de peticiones curl reales (`s08/curl_crud.log`); los escenarios del Producto 2 se ejecutaron desde la aplicación y se contrastaron con el logcat de Ktor.

Observaciones respecto a la guía, verificadas en el servidor:

- El DELETE de PharmaSoft es una baja lógica (estado = false). La segunda eliminación del mismo id no devuelve 404 sino 409 Conflict con el mensaje «El producto ... ya se encuentra inactivo»; la app lo traduce a `ErrorApi.Conflicto`.
- El precio 0 o negativo lo detecta antes la validación local del formulario. Para provocar el 400 del servidor sobre `precioError` se envió 0.001, que supera la validación local pero el servidor rechaza («El precio debe ser mayor que cero»). El precio 0 enviado directamente por curl también devuelve 400.
- La app lista con `tamano=20` (valor por defecto de `ProductoApi`); la matriz usa `tamano=10` como pide la guía.
- El escenario 7 se reprodujo reduciendo temporalmente `requestTimeoutMillis` a 1 ms; el valor original (15 000 ms) fue restaurado.
- No se incluyen capturas ni pruebas de iOS: el entorno de desarrollo es Windows y no dispone de macOS/Xcode. Todas las capturas son de Android; la lógica compartida (`commonTest`) no depende de la plataforma.
- El escenario 3 se reprodujo borrando físicamente en la base de pruebas un producto de prueba creado por la app, de modo que la lista de la app quedó desactualizada y el PUT devolvió 404.

Producto 1: Matriz de operaciones CRUD REST (PharmaSoft)

Recurso `/api/v1/productos`. El código recibido proviene de `s08/curl_crud.log`.

#	Método y ruta	Parámetros / Payload	DTOs	HTTP esperado	HTTP recibido	Resultado observado
1	GET <code>/api/v1/productos?pagina=0&tamano=10</code>	<code>pagina=0, tamano=10</code>	<code>PaginaResponseDto</code> < <code>ProductoResponseDto</code> >	200 OK	200 OK	Página con contenido, <code>totalElementos</code> y <code>totalPaginas</code>
2	GET <code>/api/v1/productos/{id}</code>	<code>id = 21</code>	<code>ProductoResponseDto</code>	200 OK	200 OK	Producto 21 «Loratadina 10mg»
3	POST <code>/api/v1/productos</code>	<code>{"nombre":"Loratadina 10mg","precio":8.9,"stock":30,"estado":true,"categoriaId":1}</code>	<code>ProductoRequestDto</code> → <code>ProductoResponseDto</code>	201 Created	201 Created	Producto creado con id 21
4	PUT <code>/api/v1/productos/{id}</code>	<code>{"nombre":"Loratadina 10mg Forte","precio":9.9,"stock":35,...}</code>	<code>ProductoRequestDto</code> → <code>ProductoResponseDto</code>	200 OK	200 OK	Nombre, precio y stock actualizados; <code>fechaModificacion</code> asignada
5	DELETE <code>/api/v1/productos/{id}</code>	<code>id = 21</code>	Sin cuerpo	204 No Content	204 No Content	Respuesta vacía; <code>ProductoApi.eliminar</code> no llama a <code>body()</code> , no se deserializa JSON

Extracto del registro de peticiones (`curl_crud.log`):

```
### POST /api/v1/productos -> HTTP 201
{"id":21,"nombre":"Loratadina 10mg","precio":8.9,"stock":30,"estado":true,"categoriaId":1,...}
### PUT /api/v1/productos/21 -> HTTP 200
{"id":21,"nombre":"Loratadina 10mg Forte","precio":9.9,"stock":35,...,"fechaModificacion":"2026-10-01T21:00:36.85"}
```

DELETE /api/v1/productos/21 -> HTTP 204 (cuerpo vacío)

Evidencias en la aplicación (emulador Pixel 8):

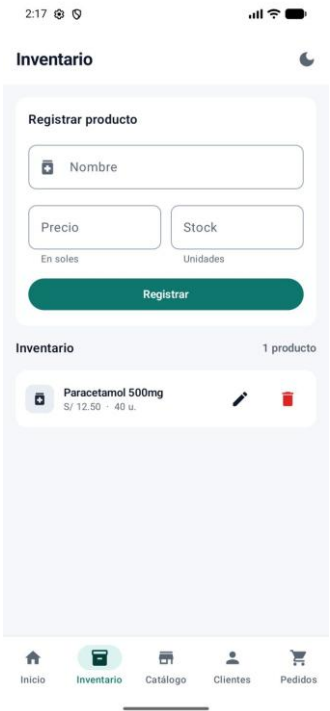


Figura 1. GET paginado: inventario cargado desde PharmaSoft (Fase.ConProductos).

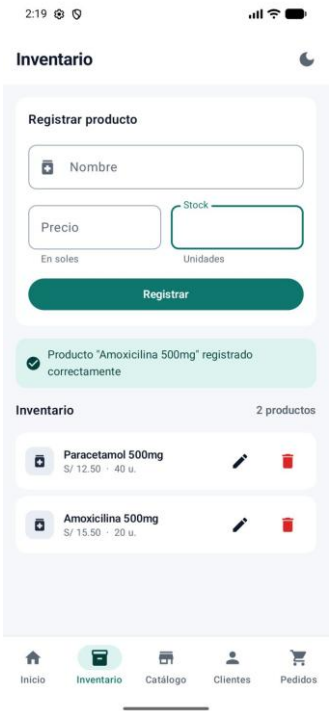


Figura 2. POST 201: «Amoxicilina 500mg» registrado y lista recargada.



Figura 3. Formulario precargado para editar el producto (PUT).



Figura 4. PUT 200: stock actualizado de 20 a 35 unidades.

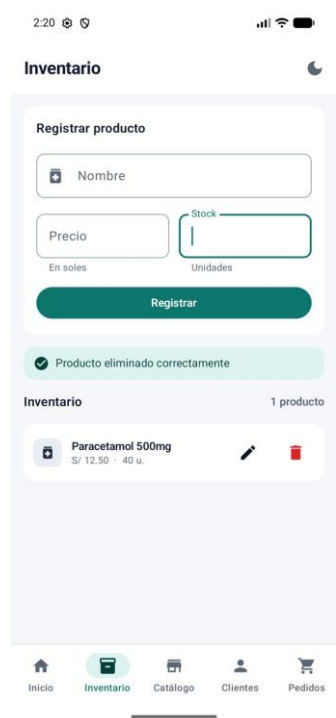


Figura 5. DELETE 204: el producto desaparece del listado.

Producto 2: Bitácora de 8 escenarios de fallo controlados

Resumen de la taxonomía ErrorApi aplicada en cada escenario:

Esc.	Disparador	HTTP / excepción	ErrorApi	Mensaje en la UI
1	POST con nombre «Ab»	400	Validacion	El nombre debe tener entre 3 y 150 caracteres (bajo nombreError)
2	POST con precio 0.001 (0 por curl)	400	Validacion	El precio debe ser mayor que cero (bajo precioError)
3	PUT sobre id inexistente	404	NoEncontrado	El producto ya no existe
4	DELETE repetido sobre el mismo id	204 y luego 409	Conflicto	El producto ... ya se encuentra inactivo
5	POST con nombre duplicado	409	Conflicto	Ya existe un producto con el nombre Paracetamol 500mg
6	Modo avión	ConnectException	SinConexion	Sin conexión con el servidor
7	requestTimeoutMillis = 1	HttpRequestTimeoutException	TiempoAgotado	El servidor tardó demasiado en responder
8	Cancelar el Scope en curso	CancellationException	(ninguno)	Sin mensaje; la corrutina se cancela limpiamente

Escenario 1: Validación del servidor (nombre corto)

Fecha/Hora	01/10/2026 · 21:16
Plataforma	Android (emulador Pixel 8) · backend PharmaSoft :8080

Pasos	Inventario → Nombre «Ab», Precio 5, Stock 1 → Registrar.
Código HTTP	400 Bad Request
ErrorResponseDto	validationErrors = {"nombre":"El nombre debe tener entre 3 y 150 caracteres"}
ErrorApi producido	ErrorApi.Validacion(porCampo = {nombre: ...})
Mensaje en la UI	«El nombre debe tener entre 3 y 150 caracteres» bajo el campo Nombre (nombreError).
Conclusión técnica	El 400 se mapea por campo; la Fase no cambia a Error y Operacion vuelve a Inactiva.



Figura 6. Escenario 1: error 400 bajo el campo Nombre.

Escenario 2: Precio inválido

Fecha/Hora	01/10/2026 · 21:17
Plataforma	Android (emulador Pixel 8) · backend PharmaSoft :8080
Pasos	Nombre «Prueba Precio», Precio 0.001, Stock 1 → Registrar. (El valor 0 lo frena la validación local; por curl también devuelve 400.)
Código HTTP	400 Bad Request
ErrorResponseDto	validationErrors = {"precio":"El precio debe ser mayor que cero"}
ErrorApi producido	ErrorApi.Validacion(porCampo = {precio: ...})
Mensaje en la UI	«El precio debe ser mayor que cero» bajo el campo Precio (precioError).
Conclusión técnica	El servidor es la fuente de verdad: lo que la validación local deja pasar, el 400 lo corrige por campo.

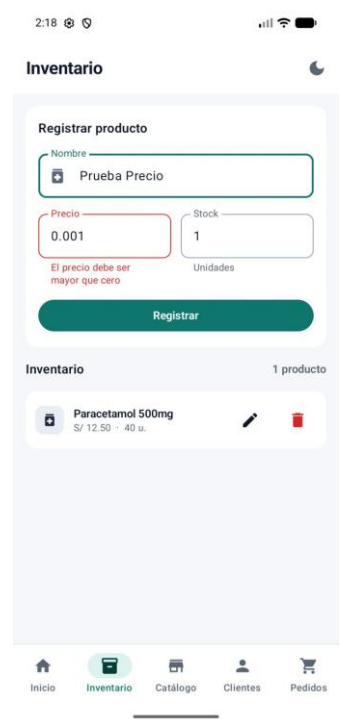


Figura 7. Escenario 2: error 400 bajo el campo Precio.

Escenario 3: Recurso inexistente

Fecha/Hora	01/10/2026 · 21:22
Plataforma	Android (emulador Pixel 8) · backend PharmaSoft :8080
Pasos	Crear «Temporal S3», borrarlo físicamente en la base de pruebas, editarlo en la app (lista desactualizada) y pulsar Guardar cambios.
Código HTTP	404 Not Found (PUT /api/v1/productos/25)
ErrorResponseDto	message = "Producto no encontrado con id: ..." (mismo formato que para el id 999999)
ErrorApi producido	ErrorApi.NoEncontrado
Mensaje en la UI	«El producto ya no existe»
Conclusión técnica	El 404 llega como Operacion.Fallida; el formulario y la lista se conservan.

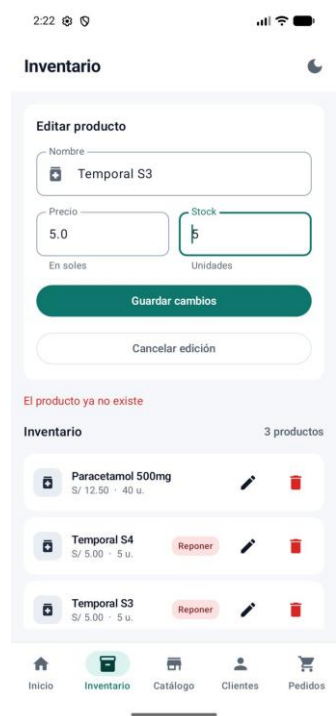


Figura 8. Escenario 3: 404 traducido a «El producto ya no existe».

Escenario 4: Doble eliminación

Fecha/Hora	01/10/2026 · 21:21
Plataforma	Android (emulador Pixel 8) · backend PharmaSoft :8080
Pasos	Crear «Temporal S4»; DELETE por curl (1.ª llamada); pulsar Eliminar en la app sobre el mismo id (2.ª llamada). Registro en s08/doble_delete.log.
Código HTTP	1.ª: 204 No Content · 2.ª: 409 Conflict (la guía esperaba 404; el backend hace baja lógica)
ErrorResponseDto	message = "El producto Temporal S4 ya se encuentra inactivo"
ErrorApi producido	ErrorApi.Conflicto(mensaje)
Mensaje en la UI	«El producto Temporal S4 ya se encuentra inactivo»
Conclusión técnica	Con baja lógica el segundo DELETE es una violación de regla de negocio (409), no un recurso inexistente.

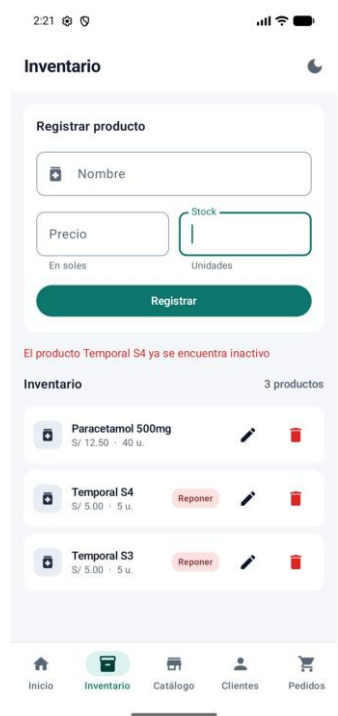


Figura 9. Escenario 4: segunda eliminación rechazada con 409.

Escenario 5: Regla de negocio / conflicto

Fecha/Hora	01/10/2026 · 21:20
Plataforma	Android (emulador Pixel 8) · backend PharmaSoft :8080
Pasos	Registrar un producto con nombre duplicado «Paracetamol 500mg».
Código HTTP	409 Conflict
ErrorResponseDto	message = "Ya existe un producto con el nombre Paracetamol 500mg"
ErrorApi producido	ErrorApi.Conflicto(mensaje)
Mensaje en la UI	«Ya existe un producto con el nombre Paracetamol 500mg»
Conclusión técnica	El mensaje del servidor se muestra tal cual; no se pierde información de negocio.

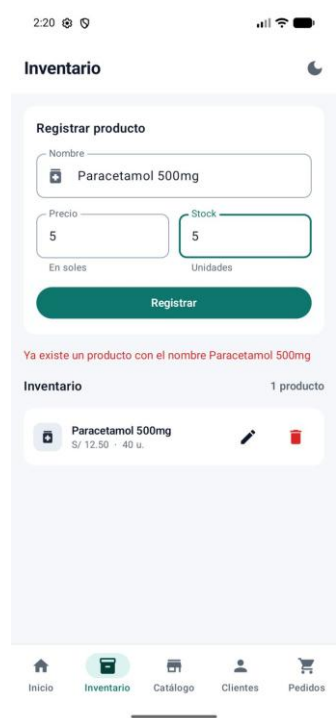


Figura 10. Escenario 5: conflicto 409 con el mensaje emergente del servidor.

Escenario 6: Servidor caído / sin conexión

Fecha/Hora	01/10/2026 · 21:22
Plataforma	Android (emulador Pixel 8) · backend PharmaSoft :8080
Pasos	Activar Modo Avión en el emulador y pulsar Eliminar sobre un producto.
Excepción	java.net.ConnectException: Failed to connect to /10.0.2.2:8080 (logcat_ktor_crud.log)
ErrorApi producido	ErrorApi.SinConexion
Mensaje en la UI	«Sin conexión con el servidor» como Operacion.Fallida. La carga inicial muestra la pantalla de error con ícono de nube tachada y botón Reintentar (ver Figura 12).
Conclusión técnica	La excepción de red se traduce sin exponer tipos de Ktor a la presentación.

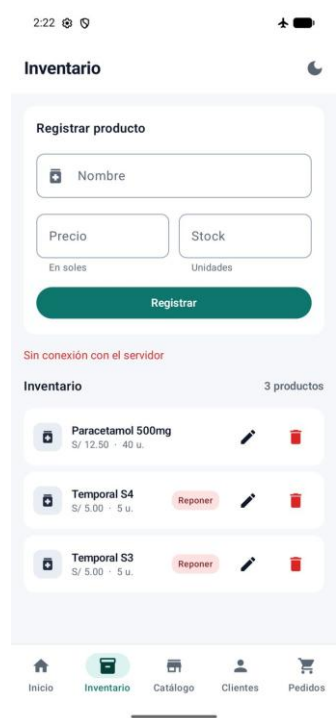


Figura 11. Escenario 6: Modo Avión activo (ícono en la barra de estado).

Escenario 7: Tiempo de espera agotado

Fecha/Hora	01/10/2026 · 21:23
Plataforma	Android (emulador Pixel 8) · backend PharmaSoft :8080
Pasos	requestTimeoutMillis = 1 en HttpClientFactory, reinstalar y abrir Inventario. Valor original restaurado después.
Excepción	HttpRequestTimeoutException: Request timeout has expired (request_timeout=1 ms), ver logcat_timeout.log
ErrorApi producido	ErrorApi.TiempoAgotado
Mensaje en la UI	«El servidor tardó demasiado en responder» con botón Reintentar.
Conclusión técnica	El timeout se clasifica antes del caso genérico y no se confunde con una cancelación de corrutina.

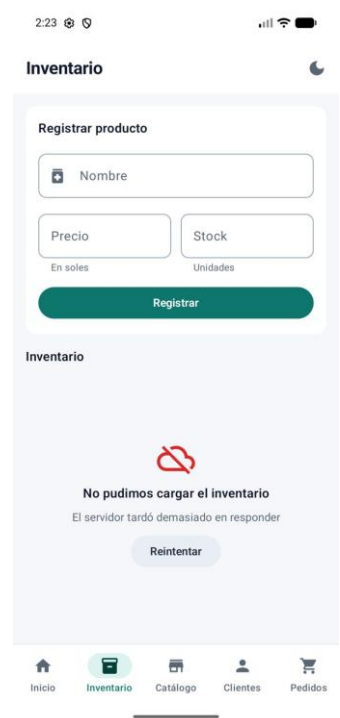


Figura 12. Escenario 7: pantalla de error con ícono de red y botón Reintentar.

Escenario 8: Cancelación de corrutina

Fecha/Hora	01/10/2026 · 22:02 (emulador) + pruebas automatizadas
Plataforma	Android (emulador Pixel 8) · backend PharmaSoft :8080
Pasos	Se congeló el contenedor Oracle (docker pause) para dejar la petición GET en vuelo. Se abrió Inventario (spinner «Cargando inventario...», Figura 13) y se salió de la app con la tecla Atrás, lo que destruye la pantalla y cancela el viewModelScope. Después se descongeló la base y se reabrió la app (Figura 14).
Excepción observada	kotlinx.coroutines.JobCancellationException: Job was cancelled; job=SupervisorJobImpl{Cancelling} (logcat_cancelacion.log). No hubo FATAL EXCEPTION.
ErrorApi producido	Ninguno. toErrorApi() y traducirErrores relanzan la CancellationException antes del catch genérico, así que no se convierte en ErrorApi.Desconocido.
Mensaje en la UI	Ninguno: no se muestra error porque la pantalla ya no existe.
Qué ocurre realmente	En esta implementación el ViewModel es de la pantalla: al salir se cancela la corrutina, Ktor registra la petición como cancelada y el proceso sigue vivo. Al volver, se crea un estado nuevo y la carga termina en Fase.ConProductos con la lista real; no queda estado corrupto.
Conclusión técnica	Verificado en el emulador y con ErrorApiTest (4/4). Distinto del timeout (escenario 7): el timeout es una IOException que sí se traduce a TiempoAgotado; la cancelación es cooperativa y debe propagarse, porque tragarla dejaría la corrutina colgada.

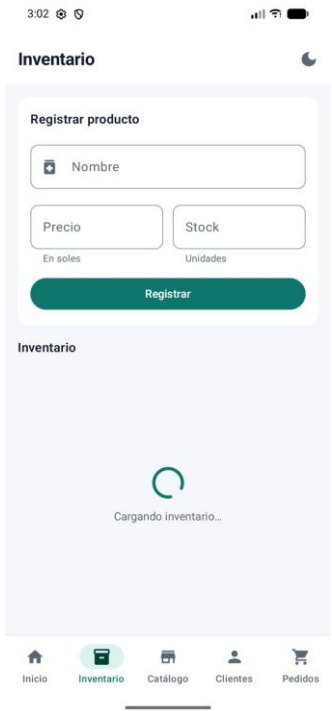


Figura 13. Escenario 8: petición en vuelo con la base de datos congelada.

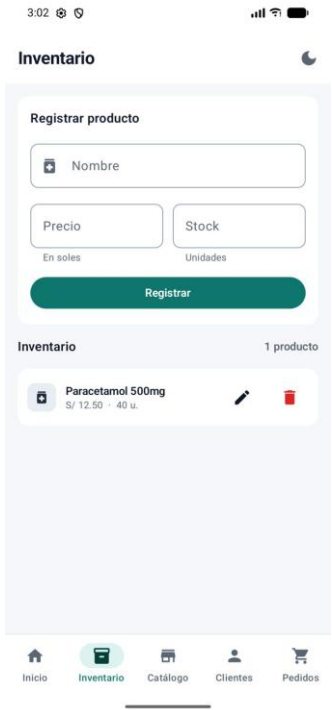


Figura 14. Escenario 8: tras reabrir y descongelar la base, la carga termina con normalidad.

Producto 3: Pruebas automatizadas multiplataforma (commonTest)

Comando ejecutado: `./gradlew testAndroidHostTest assembleDebug` → BUILD SUCCESSFUL (s08/gradle_build.log).

Requerimiento	Prueba	Qué verifica
Prueba 1: carga exitosa	ProductoViewModelTest.laCargaPasaDeCargandoAConProductos	Con la lista retenida la fase es Fase.Cargando; al liberarla pasa a Fase.ConProductos con la lista esperada.

Prueba 2: listado vacío	ProductoViewModelTest.arrancaEnSinProductosCuandoElInventarioEstaVacio	Con repositorio vacío la fase termina en Fase.SinProductos.
Prueba 3: error de validación	losErroresDeValidacionCaenEnElFormularioNoEnLaFase y elErrorDeValidacionDelServidorCaeBajoCadaCampo	Los mensajes quedan en nombreError, precioError o stockError sin cambiar la fase a Fase.Error.
Prueba 4: eliminación	ProductoViewModelTest.eliminarPasaPorEnCursoEliminarYVuelveAlInactiva	Durante eliminar() el estado es Operacion.EnCurso(Tipo.Eliminar); al terminar vuelve a Operacion.Inactiva.
Escenario 8	ErrorApiTest (4 pruebas)	CancellationException se relanza; cancelar el Job no produce ErrorApi; conexión fallida → SinConexion; ConnectTimeoutException → TiempoAgotado.

Resultados por clase de prueba (XML de Gradle):

Clase	Pruebas	Fallos
SharedCommonTest	1	0
SharedLogicAndroidHostTest	1	0
data.mapper.ProductoMapperTest	2	0
data.repository.ClienteRepositorioEnMemoriaTest	2	0
data.repository.ProductoRepositorioEnMemoriaTest	2	0
di.AppModuleTest	4	0
domain.error.ErrorApiTest	4	0
domain.model.ClienteTest	4	0
domain.model.ProductoTest	5	0
domain.usecase.ListarProductosUseCaseTest	2	0
domain.usecase.RegistrarClienteUseCaseTest	8	0
domain.usecase.RegistrarProductoUseCaseTest	11	0
presentation.cliente.ClienteViewModelTest	4	0
presentation.producto.ProductoViewModelTest	11	0
TOTAL	61	0

Salida de la ejecución:

```
> Task :shared:testAndroidHostTest
> Task :androidApp:assembleDebug
BUILD SUCCESSFUL in 34s
70 actionable tasks: 70 executed
```

Producto 4: Evidencias de ejecución y registro de Ktor

Registro del cliente Ktor (logcat filtrado por «HttpClient»; s08/logcat_ktor_crud.log):

```
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20
HttpClient: RESPONSE: 200
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos
HttpClient: RESPONSE: 400
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos
HttpClient: RESPONSE: 400
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos
HttpClient: RESPONSE: 201
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20
HttpClient: RESPONSE: 200
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos/23
HttpClient: RESPONSE: 200
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20
HttpClient: RESPONSE: 200
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos/23
HttpClient: RESPONSE: 204
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20
HttpClient: RESPONSE: 200
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos
HttpClient: RESPONSE: 409
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos
HttpClient: RESPONSE: 201
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20
HttpClient: RESPONSE: 200
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos
HttpClient: RESPONSE: 201
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20
HttpClient: RESPONSE: 200
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos/24
HttpClient: RESPONSE: 409
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos/25
HttpClient: RESPONSE: 404
HttpClient: REQUEST: http://10.0.2.2:8080/api/v1/productos/25
HttpClient: REQUEST http://10.0.2.2:8080/api/v1/productos/25 failed with exception: java.net.ConnectException:
Failed to connect to /10.0.2.2:8080
```

Registro de la excepción de timeout y de conexión:

```
HttpClient: REQUEST http://10.0.2.2:8080/api/v1/productos/25 failed with exception: java.net.ConnectException:
Failed to connect to /10.0.2.2:8080
HttpClient: REQUEST http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20 failed with exception:
java.util.concurrent.CancellationException: Reque
```

Commits de la actividad (rama feature/ktor-client):

Commit	Mensaje	Enlace
8f716de	docs(s08): ajustar anchos de tabla y tamaño de capturas en el informe	https://github.com/fabianrodriguez33/Pharma-Mobil/commit/8f716de
93a73c6	docs(s08): completar informe en Word de actividad autónoma 08 y actualizar readme	https://github.com/fabianrodriguez33/Pharma-Mobil/commit/93a73c6
3a1f5e5	docs(s08): agregar documento de entrega de la guía 08	https://github.com/fabianrodriguez33/Pharma-Mobil/commit/3a1f5e5
329eca4	test(s08): cubrir el mapper de producto (inactivos y request)	https://github.com/fabianrodriguez33/Pharma-Mobil/commit/329eca4
c53282f	fix(s08): enviar valores por defecto en el body y omitir productos inactivos; evidencias e2e	https://github.com/fabianrodriguez33/Pharma-Mobil/commit/c53282f
f4e321a	feat(s08): implementar crud rest completo con ktor y estados	https://github.com/fabianrodriguez33/Pharma-Mobil/commit/f4e321a

	de ui	Mobil/commit/f4e321a
--	-------	----------------------

README.md actualizado con las secciones «CRUD REST PharmaSoft» y «Manejo de Errores».